Project Notes: Restaurant Management and Online Booking System

Study notes for the group. Updated after every completed step.

Progress: Phase 1 (Project Setup) ✅ · Phase 2 (Backend Foundation) ✅ · Next: Phase 3 (Frontend Foundation)

Contents

1. Project Overview
2. Business Rules
3. Current Folder Structure
4. How a Request Travels Through the Backend
5. Step-by-Step Log
6. API Reference
7. HTTP Status Codes We Use
8. Errors We Hit and How We Fixed Them
9. Possible Instructor Questions
10. Glossary

---

1. Project Overview

What the system does

A web application for one restaurant that handles:

- **Online reservations**: customers book a table for a date and time; staff confirm it; the customer dines; staff mark it completed.
- **Restaurant management**: staff manage tables (capacity, location, availability) and monitor reservations.
- **Accounts and users**: registration, login, profiles, password recovery; an admin manages all accounts and roles.
- **Archive**: finished records (completed/cancelled reservations, inactive users) move out of the active lists but can still be searched, viewed, and restored.

Technology stack (MERN)

Letter	Technology	Role in our project
M	MongoDB	Database (Phase 9; temporary JavaScript arrays for now)
E	Express.js	Backend framework: receives requests, sends responses
R	React (with Vite)	Frontend: the pages users see

Letter	Technology	Role in our project
N	Node.js	Runs the backend JavaScript

Target client (sample/reference only)

Hapag Filipino Kitchen, a fictional mid-sized Filipino casual-dining restaurant in Quezon City, used as a reference to define roles, rules, and workflow.

> We say "sample/reference client". We do **not** claim a real restaurant approved or uses the system.

User roles

Role	Who	Can	Cannot
ADMIN	Owner / manager	Everything: manage users and roles, tables, all reservations, archive and restore	Delete their own admin account
STAFF	Front desk / host / service crew	View, confirm, reject, edit, and complete reservations; manage table availability; view customer info; archive reservations	Manage users or roles; restore archived users
CUSTOMER	Diners	Register, log in, edit profile, check availability, book, view and cancel **own** reservations	See other customers' data; use staff/admin pages

Planned demo accounts: **1 Admin + 3 Staff + 6 Customers = 10 users**.

> Roles take effect in **Phase 4 (Authentication)**. Until then, every endpoint is open.

Major modules

1. Account Management: register, login, logout, profile, password recovery
2. User Management: admin CRUD, roles, active/inactive status
3. Table Management
4. Reservation (core transaction): booking, availability, conflict prevention, statuses
5. Staff/Admin Dashboard
6. Archive: archive, view, search/filter, restore

Overall workflow

```
Customer registers / logs in
      ↓
Checks availability (date + time + guests) → picks a free table
      ↓
Submits reservation ──────────────────→ PENDING
      ↓
Staff reviews ─── rejects ──────────→ CANCELLED
      ↓ confirms
CONFIRMED ─── customer cancels ─────→ CANCELLED
      ↓ customer dines
```

```
Staff marks ──────────────────────────────→ COMPLETED
      ↓
Admin/Staff archives (completed/cancelled only) → ARCHIVE → view / search /
restore
```

---

2. Business Rules

All of these live in `server/config/businessRules.js`, so they are easy to find and change.

Seatings (time slots)

Each seating lasts **2 hours**, so slots never overlap.

Seating	Time
Lunch 1	11:00 – 13:00
Lunch 2	13:00 – 15:00
Afternoon	15:00 – 17:00
Dinner 1	17:00 – 19:00
Dinner 2	19:00 – 21:00

> We first planned hourly slots, but a 7:00 PM party would still be eating when an 8:00 PM party arrived at the same table. Two-hour seatings fix this and keep the conflict rule simple.

Booking rules

Rule	Value
Party size	1 to 10 guests
Booking window	Today up to **30 days** ahead
Past dates	Not allowed
Today	Only time slots that have not passed yet
Table must fit the party	`guests ≤ table capacity`
Table must be in service	Table `status` must be `"available"`
New reservation status	Always **Pending** (the customer cannot choose it)

The conflict rule

> **A table can have only ONE active reservation (Pending or Confirmed) per date + time slot.**
> Cancelled reservations release the table. Completed ones are in the past.

Status workflow

From	Can change to	Who
Pending	Confirmed, Cancelled (= rejected)	Staff/Admin
Pending	Cancelled	Customer (own booking)
Confirmed	Completed, Cancelled	Staff/Admin
Confirmed	Cancelled	Customer (own booking)
Cancelled	nothing (final)	—
Completed	nothing (final)	—

Written in code as:

```
const ALLOWED_STATUS_CHANGES = {
  Pending: ["Confirmed", "Cancelled"],
  Confirmed: ["Completed", "Cancelled"],
  Cancelled: [],
  Completed: [],
};
```

Time rules for status changes

- Cannot **confirm** a reservation whose schedule has already passed.
- Cannot **complete** a reservation before its scheduled time.
- A customer cannot **cancel** after the schedule has started.

Editing rules

- Only **Pending/Confirmed** reservations that have **not started** can be edited.
- Only **Staff/Admin** edit; customers cancel and rebook.
- Status cannot be changed through edit (it has its own endpoint).
- The edited result must pass the same validation and conflict check as a new booking.

Table rules

- Table numbers are unique.
- Capacity is 1 to 20.
- Tables are **never deleted**; they are set to `"unavailable"` instead, because deleting a table would break the history of reservations that used it.

---

3. Current Folder Structure

```
resto-reservation-system/
├── docs/
│   └── PROJECT_NOTES.md            ← this file
├── server/                         ← Node.js + Express backend (port 5000)
```

```
│   ├── config/
│   │   └── businessRules.js        restaurant rules (slots, limits, statuses)
│   ├── controllers/
│   │   ├── tableController.js      logic for /api/tables
│   │   └── reservationController.js logic for /api/reservations
│   ├── data/                       TEMPORARY in-memory data (→ MongoDB in Phase
9)
│   │   ├── tables.js               12 tables
│   │   └── reservations.js         6 sample reservations
│   ├── middleware/
│   │   └── errorMiddleware.js      404 handler + error handler
│   ├── requests/                   REST Client test files (not app code)
│   │   ├── tables.http
│   │   ├── reservations.http
│   │   ├── reservationStatus.http
│   │   └── reservationEdit.http
│   ├── routes/
│   │   ├── tableRoutes.js          URL → controller function
│   │   └── reservationRoutes.js
│   ├── utils/
│   │   └── dateUtils.js            date/time helpers
│   ├── .env                        PORT=5000
│   ├── package.json
│   └── server.js                   entry point
└── client/                         ← React frontend made with Vite (port 5173)
    ├── src/
    │   ├── App.jsx                 backend connection test page
    │   ├── index.css               minimal global styles
    │   └── main.jsx                starts React
    ├── index.html
    └── package.json
```

Dependencies and why we installed them

Package	Where	Why
`express`	server	Web framework that receives requests and sends responses
`cors`	server	Lets the React app (port 5173) call the API (port 5000)
`dotenv`	server	Reads settings (port, later database URL and secrets) from `.env`
`nodemon` (dev only)	server	Restarts the server automatically when a file is saved
`react`, `react-dom`, `vite`	client	Installed by the Vite React template

Tool: REST Client (VS Code extension by Huachao Mao) for testing API requests from `.http` files.

How to run the project (two terminals)

Terminal	Folder	Command	URL
1: Backend	`resto-reservation-system\server`	`npm run dev`	http://localhost:5000

Terminal	Folder	Command	URL
2: Frontend	`resto-reservation-system\client`	`npm run dev`	http://localhost:5173

---

4. How a Request Travels Through the Backend

```
React / REST Client
      │  e.g. POST /api/reservations
      ▼
server.js            → runs middleware (cors, express.json), then sends the
      │                request to the matching route file
      ▼
routes/              → matches URL + method to a controller function
      │                (POST "/" → createReservation)
      ▼
controllers/         → THE LOGIC: read input, validate, check rules,
      │                change data, send response
      ▼
data/                → temporary arrays in memory (MongoDB in Phase 9)
      │
      ▼
JSON response        → { success, message, count, data } or { success: false,
message, errors }
```

If no route matches, the `notFound` middleware returns a 404 JSON message. If code throws an error, `errorHandler` returns a clean JSON error instead of crashing.

Why the layers matter

Only the **controllers** touch the data. In Phase 9 we replace `data/*.js` with Mongoose models and change a few lines in the controllers; routes, `server.js`, and React stay the same.

Where the data lives right now (important)

```
data/reservations.js (file on disk)
        │  read ONCE when the server starts
        ▼
reservations array (in RAM)  ◄── all bookings, edits, cancellations happen here
        │
        ▼
Server restarts → RAM is wiped → array is read again from the file
```

That is why:

- the file never changes when we test requests;
- saving any backend file (nodemon restart) resets the data;
- we **need MongoDB**, because a real system must remember changes.

5. Step-by-Step Log

STEP 1: Backend setup and first Express server

Goal: prove Node, npm, and Express work.

What we did

- Created `server/`, ran `npm init -y`, installed `express cors dotenv` and `nodemon` (dev).
- Added scripts in `package.json`: `"start": "node server.js"`, `"dev": "nodemon server.js"`.
- Created `.env` with `PORT=5000`.
- Created `server.js` with one test route: `GET /api/health`.

Key logic `dotenv` loads `PORT`. Express creates the app. `cors()` and `express.json()` run on every request. `app.listen` starts the server. All API routes start with `/api/...` to keep backend URLs separate from frontend pages.

Test: http://localhost:5000/api/health → `{"status":"ok", ...}`

STEP 2: React frontend and frontend-to-backend connection

Goal: prove React can talk to Express.

What we did

- Created `client/` with `npm create vite@latest client -- --template react`, then `npm install`.
- Removed Vite demo files (`App.css`, logos).
- Replaced `App.jsx` with a page that calls `/api/health` and shows the result.

Key logic When the page loads, `useEffect` (with `[]`) runs once and uses `fetch` to call the backend. The JSON reply is stored with `setHealth`, React re-renders, and the data appears. If the backend is down, `.catch` stores an error that shows in red.

Pattern used everywhere later: **request → response → state → re-render**.

Works only because the backend has `app.use(cors())`; without it, the browser blocks calls from port 5173 to port 5000.

Test: http://localhost:5173 shows "Connected to backend!". Stopping the backend shows the red error.

STEP 3: Backend structure and read-only Tables API

Goal: organize the backend into layers; list tables.

What we did

- Created folders `data/`, `controllers/`, `routes/`, `middleware/`.
- `data/tables.js`: 12 tables (Table 11 set to `"unavailable"` for testing).
- `GET /api/tables` with filters `?status=` and `?minCapacity=`.

- `GET /api/tables/:id`.
- `errorMiddleware.js`: `notFound` (404) and `errorHandler` (500/other).

Key logic

- We use `_id` (not `id`) because MongoDB uses `_id`, so the frontend will not need changes later.
- Table `status` means *in service or not*. Whether a table is free at a specific date/time is computed from reservations.
- Every response has the same shape: `{ success, count, data }`.
- `notFound` must be placed **after** all routes, because Express checks top to bottom.

Presentation line:

> "When React requests `/api/tables?minCapacity=6`, `server.js` passes it to `tableRoutes`, which calls `getAllTables`. The controller reads the filters, validates them, filters the list, and returns JSON. If the URL doesn't exist or something fails, the error middleware returns a clean JSON error instead of crashing."

STEP 4: Create and update tables with validation

Goal: add POST/PUT with input validation.

What we did

- `POST /api/tables` and `PUT /api/tables/:id`.
- `validateTable()` helper (required fields for create, optional for update).
- Installed the REST Client extension; created `requests/tables.http`.

Key logic

- Validation: table number is a positive whole number, capacity 1–20, location required, status only `available`/`unavailable`.
- Table numbers must be unique (409 Conflict if taken).
- New records are built from **allowed fields only**, so extra fields sent by a client are ignored.
- PUT updates only the fields that were sent.
- `req.body || {}` because Express 5 leaves `req.body` undefined when no JSON is sent.
- **No DELETE**: tables with reservation history should not disappear.

Presentation line:

> "POST validates the input first, checks that the table number isn't used, builds the table from allowed fields only, saves it, and returns 201. PUT updates only the fields sent and uses the same validation. We use 400 for invalid input, 404 when not found, and 409 for conflicts."

STEP 5: Reservations, availability, and conflict prevention (core)

Goal: let customers book without double-booking.

What we did

- `config/businessRules.js` (slots, guest limits, booking window, statuses).
- `utils/dateUtils.js` (`getToday`, `addDays`, `getCurrentTime`, `isValidDateString`).
- `data/reservations.js` with sample bookings in different statuses.
- Endpoints: `GET /api/reservations`, `GET /api/reservations/availability`, `GET /api/reservations/:id`, `POST /api/reservations`.

Booking flow (POST)

```
1. Validate input (name, email, PH mobile, date, time slot, guests, table)
2. Check date (not past, max 30 days ahead, slot not passed today)
3. Check table (exists, in service, big enough)
4. CONFLICT CHECK (same table + date + time with Pending/Confirmed?)
5. Save with status "Pending"
6. Return 201 + reservation
```

The conflict check

```
const isTableBooked = (tableId, date, time) =>
  reservations.some(
    (r) =>
      r.table === tableId &&
      r.date === date &&
      r.time === time &&
      ACTIVE_STATUSES.includes(r.status)
  );
```

`.some()` returns `true` as soon as ONE reservation matches ALL four conditions.

Sample data used to prove it

Record	Purpose
r1: Maria Santos, Table 5, Oct 10, 19:00, **Confirmed**	The requirement example; a second booking for the same slot gets **409**
r4: Paolo Garcia, Table 6, Oct 10, 19:00, **Cancelled**	Proves cancelled bookings do **not** block the table

Design decisions

- Dates are stored as `"2026-10-10"` and times as `"19:00"` (text). This format compares correctly as text and avoids timezone bugs.
- `table` stores the table `_id`; responses replace it with full table details (`withTableDetails`), the same thing MongoDB's `populate()` will do.
- `/availability` route must come **before** `/:id`, or Express would treat "availability" as an ID.
- Capacity comes from the **table data**, never from the request. The request sends `guests` (party size); the server compares `guests` with the stored `capacity`. Never trust the client for business data.

Presentation line:

> "Before saving a booking, the server looks for any reservation with the same table, date, and time slot whose status is still active (Pending or Confirmed). If one exists, it rejects the booking with 409 Conflict. Cancelled reservations aren't counted, so cancelling frees the table. Each slot is a 2-hour seating that never overlaps another, so matching the exact slot is enough. The availability endpoint uses the same check, so customers only see free tables."

STEP 6: Status workflow (confirm, reject, cancel, complete)

Goal: move reservations through their life after creation.

What we did

- Added `ALLOWED_STATUS_CHANGES` to `businessRules.js`.
- Added r6 (Ramon Bautista, past Confirmed booking) so we could test completing.
- `PATCH /api/reservations/:id/status` (Staff/Admin): confirm, reject/cancel, complete.
- `PATCH /api/reservations/:id/cancel` (Customer): cancel own booking.
- Helpers: `hasStarted()`, `validateReason()`.
- Cancelled records store `cancelledBy` (`staff`/`customer`) and `cancellationReason`; changes store `updatedAt`.

Key logic

- The server looks up the current status in `ALLOWED_STATUS_CHANGES` and checks whether the new status is in its list.
- Time rules (cannot confirm a past booking, cannot complete a future one, customer cannot cancel after start).
- Customers have a **separate** endpoint because they may only cancel.
- Cancelling needs no "free the table" code: the conflict check only counts Pending/Confirmed, so the table becomes available automatically.
- **PATCH vs PUT:** PATCH changes one part (status); PUT edits a record's details.

Test scenarios (reservationStatus.http)

Purpose	Requests
Staff confirms	#1 (r2 Juan)
Staff rejects	#3 (r3 Angela)
Staff completes	#5 (r6 Ramon)
Customer cancels; table freed	#9, #10 (r1 Maria)
Blocked wrong actions	#2, #4, #6, #7, #8, #11, #12

Presentation line:

> "Each reservation follows a fixed workflow stored in one object. Before changing a status, the server checks whether the change is allowed. We also apply time rules, like not completing a reservation

> before the guest's schedule. Customers have a separate cancel endpoint, and we record who cancelled and why."

Connection to requirements: the **20 completed transactions** are reservations that went Pending → Confirmed → Completed, like r6.

STEP 7: Editing a reservation

Goal: staff can change date, time, guests, table, or details (e.g., a customer calls to move their booking).

What we did

- `PUT /api/reservations/:id` (Staff/Admin).
- Moved field checks into one shared helper, `validateReservationData()`, used by both create and edit.
- `isTableBooked()` got a 4th parameter, `excludeId`.

Edit flow

```
1. Find the reservation                      → 404 if missing
2. Pending/Confirmed and not started?        → 400 if not
3. "status" in body?                         → 400 (use /status)
4. Keep only editable fields that were sent  → 400 if none
5. MERGE: current data + changes
6. Validate the merged result (same rules as a new booking)
7. Check the table (exists, in service, big enough)
8. Conflict check, IGNORING this reservation itself → 409 if taken
9. Save changes + updatedAt
```

The self-conflict problem Editing only the special request of r1 (Table 5, Oct 10, 19:00) would make the conflict check find **r1 itself** and wrongly reject it. Fix:

```
isTableBooked(updated.table, updated.date, updated.time, reservation._id)
// checks every reservation EXCEPT the one being edited
```

Where the edit actually happens `const updated = { ...reservation, ...changes }` is only a **draft** for validation. The real edit happens when assigning to `reservation.table = ...` etc. `reservation` points to the real object inside the in-memory array, so the array changes directly (the file does not).

Presentation line:

> "Staff send only the fields that changed. The server merges them with the existing data and validates the full result using the same function as new bookings. It then runs the conflict check but ignores the reservation being edited, so it doesn't conflict with itself. Status can't be changed through edit, so the workflow rules can't be bypassed."

---

6. API Reference

Base URL: `http://localhost:5000/api`

Health

Method	URL	Purpose
GET	`/health`	Check that the API is running

Tables

Method	URL	Purpose	Future access (Phase 4)
GET	`/tables`	List tables (`?status=`, `?minCapacity=`)	Everyone
GET	`/tables/:id`	One table	Everyone
POST	`/tables`	Create table	Staff, Admin
PUT	`/tables/:id`	Update table	Staff, Admin

Reservations

Method	URL	Purpose	Future access (Phase 4)
GET	`/reservations`	List (`?date=`, `?status=`, `?table=`)	Staff, Admin
GET	`/reservations/availability?date=&time=&guests=`	Free tables for a schedule	Everyone
GET	`/reservations/:id`	One reservation	Staff, Admin, owner
POST	`/reservations`	Make a booking	Customer
PUT	`/reservations/:id`	Edit details	Staff, Admin
PATCH	`/reservations/:id/status`	Confirm / reject / complete	Staff, Admin
PATCH	`/reservations/:id/cancel`	Cancel own booking	Customer (owner)

Response format

```
{ "success": true, "count": 3, "data": [ ... ] }
{ "success": true, "message": "Reservation updated", "data": { ... } }
{ "success": false, "message": "Validation failed", "errors": [ "..." ] }
```

---

7. HTTP Status Codes We Use

Code	Meaning	Example in our system
200 OK	Success	Listing tables, confirming a reservation
201 Created	New record saved	New table or reservation
400 Bad Request	Invalid input or rule broken	Past date, too many guests, invalid status change
404 Not Found	Record or route doesn't exist	`/api/reservations/r999`, `/api/banana`
409 Conflict	Clashes with existing data	Table already booked; table number already used
500 Server Error	Unexpected bug	Caught by `errorHandler`

---

8. Errors We Hit and How We Fixed Them

Error	Cause	Fix
`npm error Missing script: "dev"`	`package.json` edit not saved	Add `"dev": "nodemon server.js"` to scripts and **save**
Browser showed `{"success":true,"status":"ok"}`	Another project (`online_system`) was already using port 5000	Stop the other server, or change `PORT` in `.env`
`Failed to resolve import "./assets/react.svg"`	`App.jsx` still had Vite demo code after deleting its files	Replace `App.jsx` contents and save
POST/PUT requests all returned 404	Controller and routes files still had the old versions (not saved)	Replace both files and save
`[nodemon] app crashed` → `Cannot find module '../data/reservations'`	File was named `reservation.js` (missing "s")	Rename to `reservations.js`

Lessons

- Turn on **File** → **Auto Save**. A ● dot on a tab means unsaved.
- When nodemon says `app crashed`, scroll **up**; the real error is printed above it.
- `rs` + Enter in the nodemon terminal restarts the server manually.
- Data resets on every restart (in-memory data), so run tests in order after a fresh start.

---

9. Possible Instructor Questions

Q: How do you prevent double-booking? Before saving, the server checks for any reservation with the same table, date, and time slot whose status is Pending or Confirmed. If one exists, it returns 409 Conflict. The

availability list uses the same check.

Q: What happens to the table when a reservation is cancelled? It becomes available automatically, because the conflict check only counts Pending and Confirmed reservations.

Q: Why 2-hour time slots instead of exact times? Fixed, non-overlapping seatings make conflicts easy to detect and match how restaurants run lunch/dinner seatings. A party at 19:00 never overlaps one at 17:00.

Q: Why can't a customer set the status or table capacity? The client cannot be trusted for business data. New bookings are always Pending, and capacity is read from the restaurant's own table records.

Q: Why are tables not deleted? Old reservations refer to tables. Deleting a table would break that history, so we mark it `"unavailable"` instead.

Q: Why are dates stored as text like "2026-10-10"? This format sorts and compares correctly as text, and it avoids timezone shifts between Philippine time and UTC.

Q: Why does editing a reservation not conflict with itself? The conflict check receives the ID of the reservation being edited and skips it.

Q: Why is status changed through a separate endpoint? So the workflow rules (`ALLOWED_STATUS_CHANGES` and the time rules) always apply and cannot be bypassed by a general edit.

Q: What is middleware? Functions that run on requests before or after routes. We use `cors()`, `express.json()`, a 404 handler, and an error handler. In Phase 4 we'll add authentication and role-checking middleware.

Q: Why use `_id` in temporary data? MongoDB names its ID field `_id`. Using the same name now means the frontend doesn't change when we switch to the database.

Q: Where is the data stored right now? In JavaScript arrays in the server's memory, loaded from `data/*.js` when the server starts. Changes are lost on restart, which is why Phase 9 moves everything to MongoDB.

Q: What are the 400, 404, and 409 codes for? 400 = invalid input or rule broken, 404 = not found, 409 = conflict with existing data.

---

10. Glossary

Term	Meaning
API	The set of URLs the frontend calls to get or change data
Endpoint	One URL + method, e.g. `POST /api/reservations`
Route	Code that connects an endpoint to a controller function
Controller	Function containing the logic for an endpoint
Middleware	Function that runs on requests before/after routes
Request body	JSON data sent with POST/PUT/PATCH
Query string	Filters in the URL after `?`, e.g. `?date=2026-10-10`

Term	Meaning
URL parameter	Value inside the URL path, e.g. `r1` in `/reservations/r1` (`req.params.id`)
GET / POST / PUT / PATCH	Read / create / edit details / change one part
CORS	Browser rule that blocks calls between different origins unless the server allows them
State (React)	Data a component remembers; changing it re-renders the page
useEffect	React hook that runs code after rendering (e.g., load data once)
nodemon	Tool that restarts the server when files change
In-memory data	Data kept in RAM; lost on restart
populate()	Mongoose feature that replaces an ID with the full related record